CS240: Algorithm Design and Analysis

Submodular Optimization and Greedy Approximation for Distributed Influence Maximization

A course-scale reproduction with communication-constrained candidate sharing

Yixin Chen Bichi Zhang Yaoqin Ye

Spring 2026

Project at a Glance

- **Topic:** Influence Maximization.
- **Problem:** choose k seed nodes to maximize expected network diffusion.
- **Algorithmic lens:** monotone submodular maximization, greedy approximation, CELF lazy evaluation.
- **Extension:** GreeDI-style local candidate sharing under a communication budget.

Main empirical message

Quality, runtime, and communication form a real tradeoff.

Best theory-to-practice result

CELF cuts large-scale spread evaluations by about 74%.

Motivation and Storyline

Why influence maximization?

- Diffusion appears in viral marketing, public-health messaging, and misinformation control.
- The computational question is simple to state but hard to solve exactly.
- It gives a clean bridge from network data applications to approximation algorithms.

Presentation flow

- Formalize IC influence spread as a submodular objective.
- Compare greedy, CELF, and lightweight structural baselines.
- Study what changes when only local candidates can be communicated.

Problem Formulation

$$\max_{S \subseteq V,\ |S| \leq k} \sigma(S)$$

- $G = (V, E, p)$ is a directed graph with edge activation probability p_{uv}.
- $\sigma(S)$ is the expected number of active nodes after diffusion.
- Independent Cascade model:
  - a newly active node gets one chance to activate each inactive out-neighbor;
  - each activation attempt succeeds independently with probability p_{uv}.
- Experiments use the weighted-cascade rule $p_{uv} = 1/\text{indegree}(v)$.

Why Greedy Applies

- IC expected spread is **monotone**: adding seeds cannot reduce spread.
- It is also **submodular**: marginal gain decreases as the seed set grows.

$$\sigma(A \cup \{v\}) - \sigma(A) \geq \sigma(B \cup \{v\}) - \sigma(B)$$

$$A \subseteq B, \quad v \notin B$$

Approximation

Greedy gives $(1 - 1/e)$ for the exact objective.

Acceleration

CELF treats old marginal gains as upper bounds.

Implemented Methods

Baselines

- Random seeds
- Weighted out-degree
- PageRank

Centralized

- Monte Carlo greedy
- CELF lazy greedy
- Fair re-evaluation of final seed sets

Distributed

- $m \in \{2, 4, 8\}$ partitions
- local budget $q \in \{k, 2k, 5k\}$
- coordinator greedy on merged candidates

Implementation and Experiment Design

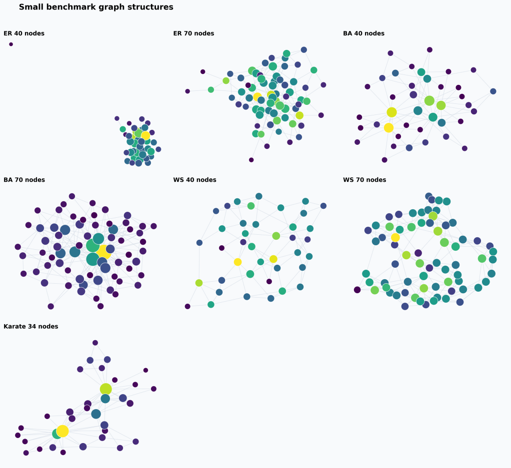
Code modules

- `simulation.py`: IC diffusion and spread estimation
- `greedy.py`: greedy and CELF
- `distributed.py`: local candidate sharing
- `experiments.py`: benchmark sweep and CSV output

Benchmarks

- Small: ER/BA/WS with $n = 40, 70$, plus Karate $n = 34$
- Large: ER/BA/WS with $n = 50, 100, 200$, plus Karate
- Seed budget: $k = 5, 6$ for small; $k = 10$ for large synthetic graphs
- Main metrics: spread, ratio to greedy, runtime, evaluations, transmitted candidates

Benchmark Graphs



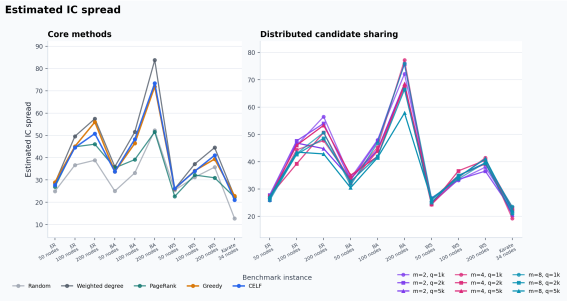
The graph families intentionally stress different structures: random connectivity, hubs, small-world locality, and a small real social network.

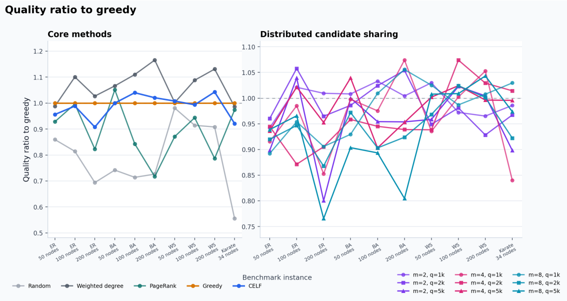
Core Results on the Large Preset

Method	Spread	Ratio	Time (s)	Eval.
Random	31.59	0.79	0.00	1.0
Weighted degree	43.71	1.07	0.01	1.0
PageRank	35.19	0.89	0.00	1.0
Greedy	40.43	1.00	4.67	1026.5
CELF	40.05	0.99	0.77	266.2

- CELF keeps essentially the same average quality as greedy while using many fewer spread evaluations.
- Degree is a strong structural baseline under weighted-cascade probabilities, especially on hub-heavy graphs.

Quality Across Graph Instances



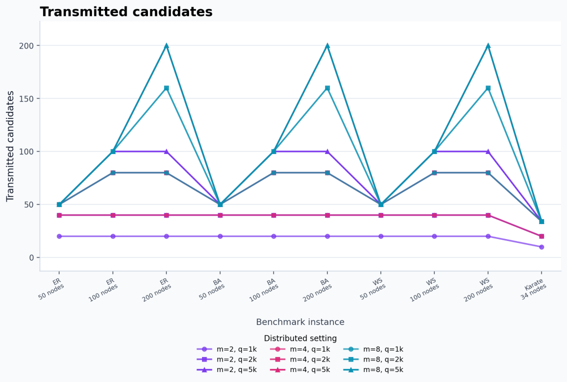


Ratios are computed after re-evaluating every final seed set with the same evaluation seed for that graph.

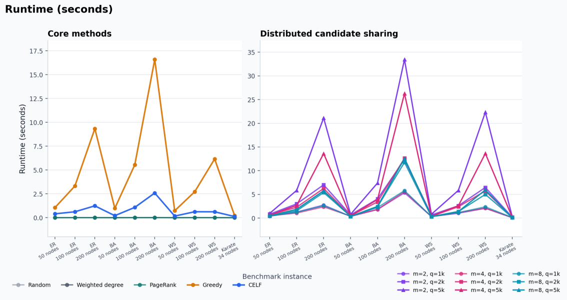
Distributed Communication Tradeoff

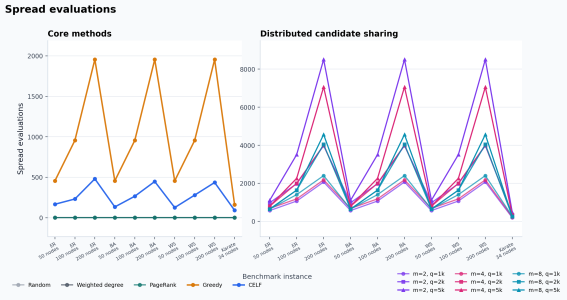
Budget	Ratio	Time	Eval.	Cand.
small, $q = k$	0.97	0.23	364	25.0
small, $q = 2k$	0.96	0.42	557	38.8
small, $q = 5k$	0.94	0.63	727	50.6
large, $q = k$	0.98	1.58	1242	41.1
large, $q = 2k$	0.96	3.36	2037	66.9
large, $q = 5k$	0.96	6.48	3044	98.4

Averaged over $m \in \{2, 4, 8\}$.



Runtime and Evaluation Counts





Distributed greedy performs extra local searches; it becomes attractive when workers run in parallel or full centralized search is constrained by data placement.

Interpretation and Limitations

What worked

- Reproduced the submodular greedy mechanism.
- CELF demonstrates a clear practical speedup.
- Distributed candidate sharing stays close to greedy quality in many cases.

What to state honestly

- Monte Carlo selection is noisy at course-scale simulation counts.
- Structural baselines can beat greedy after fair re-evaluation.
- Distributed runtime here is sequential; real speedup needs parallel workers.

Next steps: more random seeds, confidence intervals, larger real networks, deterministic partitions, and RR-set based maximum coverage.

Conclusion

- Influence maximization is a clean example of classical algorithms serving a modern network-data problem.
- IC spread is monotone submodular, so greedy approximation gives the central theoretical anchor.
- CELF shows how submodularity becomes practical acceleration.
- GreeDI-style candidate sharing exposes the cost of limited communication.
- The key lesson is not that one method always wins: graph structure, estimation accuracy, and communication budget all matter.